

EECE7352 Computer Architecture

Homework 2

Sakthivel P. Sivakumar

NUID: 002312294

Part B:

For this part of the assignment, I am using Pin Tool that is already available in the COE systems to produce the dynamic instruction mix table similar to the figure in the textbook to characterize the execution of bfs program.

```
[sakthivel@Iota EECE7352]$ ls
Insmix  bblcnt.out  bfs.c  bfs_00  bfs_02  bfs_03  insmix_00.out  insmix_02.out  insmix_03.out  pin.log  sakthivel  sample.txt
[sakthivel@Iota EECE7352]$
```

Commands to build and run the tool:

```
gcc -O0 bfs.c -o bfs_O0
```

```
cat sample.txt | /COEnet/Linux/pin-3.11/pin -t /COEnet/Linux/pin-3.11/source/tools/Insmix/obj-  
intel64/insmix.so -o insmix_O2.out -- ./bfs_O2 1
```

What is InsMix

Insmix stands for Instruction Mix tool. It's a Pintool that pin loads into the target program. What it does is, every time an instruction executes, it categorizes it (Load, Store, Branch, ALU, etc.) and counts. At the end of the run, it dumps a summary usually into insmix.out

Since Pin supports multiple host architectures, it's good to explicitly mention the type of architecture as pin needs to be compiled for the same bitness as the program that is instrumented.

In my case I am working on linux x86_64 machine, so I am using **64 bit pin tool**

Unlike normal programs compile to executables (like bfs_O0). Pintools compile to shared libraries (.so), because they are loaded by Pin into the address space of the target program. Pin is not run directly, instead:

```
pin -t obj-intel64/insmix.so -- ./bfs_O0 sample.txt
```

where,

1. pin -is the driver program,
 2. -t insmix.so = load this tool,
 3. separates Pin's args from the target app,
 4. ./bfs_O0 sample.txt = your BFS executable with its input.
-

Tabulation:

Program	Load	Store	Branches	Jump	ALU
bfs_00	53.77%	25.35%	38.44%	2.95%	-20.52%
bfs_02	50.44%	25.80%	40.37%	3.13%	-19.73%
bfs_03	50.33%	25.80%	40.37%	3.13%	-19.62%
[sakthivelps@Zeta EECE7352]\$					

This table shows the instruction mix of the given bfs program compiled with different optimization levels (O0, O2, O3) measured using Intel's Pin insmix tool.

Load and Store – reflects the memory pressure (dependency on data movement)

Branches and Jumps – indicate control flow complexities

ALU – indicated actual computation

In my table the ALU value is negative because Pin' insmix reports multiple overlapping counters.